

model-graph – an open window into a running language model

Open-source product six-pager · July 2026 · github.com/QSchlegel/model-graph

1 The problem

Language models are the most-watched black boxes in software. Developers ship agents that call tools, reason through chains, and read images – and when one goes wrong, the only observable artifact is text. Did the model not *know*? Not *look at* the right part of the prompt? Commit to a wrong answer early, or waver until the last layer? Today that question sends you to offline interpretability stacks: notebook-bound, research-oriented, hours-per-question tools built for papers, not for the loop developers actually live in – *send a prompt, watch what happens, change something, send again*.

The gap is not capability, it's **product**: model internals are cheap to capture (a dozen forward hooks) and cheap to render (a canvas), but nobody packaged them into the tool you'd reach for the way you reach for browser devtools.

2 Tenets

1. **Live-first**. Internals stream *during* generation, token by token – not post-hoc analysis of a saved run.
 2. **Honest pixels**. Nothing is rendered that wasn't measured. Missing data says so, on the canvas, never faked.
 3. **Zero-friction adoption**. One process, OpenAI-compatible – anything that speaks the OpenAI API gets instrumented for free. A mock mode runs the full UI with no GPU and no weights.
 4. **Everything is testable**. Observability that can't gate a CI run is a demo. Every run is recorded; internals carry assertions.
 5. **Tiny models are the lab**. Sub-2B models load in seconds and exhibit the same phenomena worth studying – attention sinks, settle depth, routing collapse, tool-call failures.
-

3 The product

model-graph is a self-hosted inspector with three faces on one process:

- **The chat** – a chat window whose *background is the visualization*: layers × tokens heatmaps (MLP / attention / residual writes, per-head fingerprint cells), a

logit-lens layer that dots every point where the model's prediction changes with depth (red = where it settles on the final token), per-token uncertainty, attention arcs showing where the selected token looked, and a 3D residual trajectory with the full prompt context as a linked trace. Click anything for a dashboard-grade cut-out; arrow keys walk cells; touch-ready.

- **The dashboard** – block-registry drill-down (run > layer > block > part): head panels, MoE router flow and expert utilization, SSM Δ , trace replay, deep-linkable from the chat's cut-outs.
- **The API** – OpenAI-compatible /v1/chat/completions with streaming, tool calling (parsed into spec-compliant tool_calls), and multimodal image parts – vision-language chats are fully instrumented.

Underneath: a websocket protocol any client can consume, a **curator** that records every run and gates CI with checks and tool-reasoning suites, and an **observable-ONNX** pipeline that graph-surgeries residual streams into declared outputs – so even the in-browser WebGPU engine shows real per-layer activity, serverless.

4 Customer experience

The agent developer. Maria's agent calls the wrong tool once in twenty runs. She points her existing OpenAI-SDK agent at model-graph's base URL – zero code change – and reruns. The failing case is auto-recorded; the strip shows the call token with 1.9 bits of entropy and attention parked on the system-prompt sink instead of the user's constraint. She adds the case to a curator suite with a should-call assertion, tightens the tool description, and the suite runs in CI from then on.

The interpretability tinkerer. Kai wants to *see* logit lens, not read about it. He asks "why is the sky blue" and watches the ladder: garbage → → "salinity" → ...settles on "because" at L10. He drags the residual trajectory, finds the <|im_start|> outlier, and has attention-sink intuition in an afternoon.

The educator. A transformers lecture runs live on a projector: type a prompt, watch conv and attention layers divide labour in a hybrid model, click a head-fingerprint cell. Mock mode makes the demo work on the podium laptop.

The evaluator. A team comparing tiny models runs the same reasoning suites across LFM2.5, SmolLM2 and Qwen3.5, diffing not just accuracy but settle depth, uncertainty and attention focus – internals as evaluation metrics, with regression pinning between checkpoints.

5 How it works (and what it costs)

A HookedModel wraps any HF causal or vision-language model with forward hooks (eager attention) and yields one JSON frame per token: block write norms, head norms, router weights, logit lens, attention sources, entropy/margin, and a seeded 3D projection of every residual. The API server fans frames out over a websocket (with last-run replay), records runs, and serves the UIs – one process. In-browser engines run curated ONNX models on WebGPU; the observable export exposes hidden.* outputs so client-side JS computes the same norms/trajectories from real tensors.

Cost of honesty: eager attention and per-token capture put instrumented throughput at roughly chat speed (~5 tok/s for a 1.2B model on an M-series laptop) – the right trade for an inspector.

6 Open source plan

License & posture. MIT. Single repo, batteries included; no cloud dependency, no telemetry. The docs vault and AGENTS.md make the repo legible to humans and AI agents alike – contributions are expected from both.

Contribution ladder. Suite cases and curator checks (no ML background needed) → curated browser engines → dashboard consumers for newer signals → observable-export coverage → new architecture support.

Roadmap. The reasoning program: reasoning suites with internals-based expectations, a curated failure atlas, compare-driven prompt evaluation, and fine-tuning loops gated by the same suites.

Success looks like: issues that arrive with a run id attached; suites in other projects' CI; a community-grown failure atlas; "point your agent at it" becoming the default first debugging step for local models.

FAQ. *Why not TransformerLens/nnsight?* Research libraries – powerful, offline, code-first. model-graph is a product: live, visual, OpenAI-compatible, testable. They compose. *Does my model work?* Any HF causal LM; hybrid, MoE and VL models are first-class. *Do I need a GPU?* No – mock mode for the UI, WebGPU browser engines for real inference without Python.